Ciclo de Vida DevOps

DevOps

Prof. Pedro Rocha Horchulhack
Pontifícia Universidade Católica do Paraná (PUCPR)
p.horchulhack@pucpr.br

16/10/2026



Prof. Pedro Rocha Horchulhack

Formação

- Doutorando em Informática (PUCPR, em andamento)
- Mestre em Informática (PUCPR)
- Bacharel em Ciência da Computação (PUCPR)

Atuação

- Professor em Cibersegurança
- Pesquisador em Cibersegurança
- Coordenador de Bacharelado em Ciência da Computação

Áreas de atuação

- Cibersegurança
- Computação em nuvem
- DevOps

Informações do curso

Conteúdo programático.

1. Ciclo de vida DevOps
2. Microserviços
3. Integração Contínua (CI) e Entrega Contínua (CD)
4. Ferramentas para DevOps
5. Automatização de testes, atualizações de segurança e *patches*

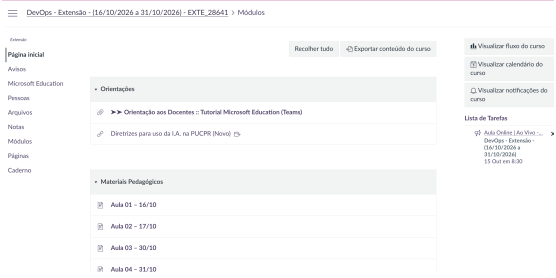
Carga horária. 32h (16h semanais)

Avaliação. Relatório técnico acerca do conteúdo lecionado ao longo do módulo. (Entrega para o dia 08/11!)

Contato

- Dúvidas referentes à quaisquer assuntos da disciplina podem ser encaminhadas por meio:
  - E-mail: `p.horchulhack@pucpr.br`
  - Teams: `p.horchulhack`
- Respondo em até 3 dias úteis 🙂

- Todos os slides, atividades e gravações das aulas estão disponíveis na nossa sala do Canvas
- `https://pucpr.instructure.com/courses/75750`



Dúvidas?

Dinâmica

Acessar a página `menti.com` e inserir o código `6553 4038`

Agenda

1. O que é o DevOps
2. Princípios
3. Atividade Prática
4. Microserviços

Agenda

1. **O que é o DevOps**
2. Princípios
3. Atividade Prática
4. Microserviços

O que é o DevOps

- Vamos imaginar o cenário onde os times de desenvolvimento, QA, operações e cibersegurança buscam a mesma coisa: garantir que a empresa/produto tenha sucesso
- Trabalhando pelo mesmo objetivo conseguimos atingir
  - Estabilidade
  - Confiabilidade
  - Disponibilidade
  - Segurança
- Nesse cenário também buscamos atender as expectativas dos clientes sem criar um caos na equipe de operações

O que é o DevOps

- Busca-se o melhor do desenvolvedor: mais qualidade e produtividade
- Ferramentas de automatização reduzem o tempo de espera entre equipes
- Com tudo isso temos o resultado da aplicação do DevOps

O que é o DevOps

- A realidade:
  - Os sistemas quebram
  - Resultados fracos e inconsistentes
  - Atritos entre desenvolvedores e operações
  - Testes e segurança ficam no final do projeto
  - Atividades críticas demandam muito trabalho manual
- Isso impacta em diversas áreas:
  - Satisfação dos colaboradores, principalmente desenvolvedores
  - Muito tempo de espera por alterações
  - Corte orçamentário

O que é o DevOps

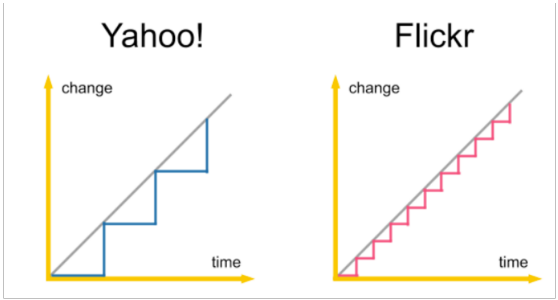
- DevOps surgiu como algo semelhante aos princípios Lean:
  - Produtividade
  - Tempo de espera do cliente
  - Qualidade do produto
  - Satisfação do cliente
- Próximo dos anos 2000, junto ao movimento ágil
  - O tempo de produção de uma funcionalidade diminuiu de meses para semanas
  - Tempo de implantação se manteve próximo de semanas e meses
  - Resultados catastróficos

O que é o DevOps

- Com o DevOps, é possível fazer implantações em horas ou minutos
- Implantar software tornou-se uma rotina
- As empresas dependem do **cadeia de valor** sobre a tecnologia

O que é o DevOps

- Exemplo:
  - **Yahoo!**: Não adotava entrega contínua[1]
  - **Flickr**: Adotou a estratégia de entrega contínua (10 por dia)
  - As entregas eram realizadas a cada alteração feita
  - Tornou o processo de integração e entrega confiáveis e consistentes



[1] Fonte: LESZKO, Rafał. Continuous delivery with Docker and Jenkins: create secure applications by building complete CI/CD pipelines. Third edition. Birmingham, UK: Packt Publishing, 2022.

O que é o DevOps

- Afinal, o que é o DevOps?
  - Uma metodologia de desenvolvimento de software
  - Mescla entre Lean Manufacturing e Agile
  - Foco em entregas rápidas e consistentes
  - Melhor interação entre Desenvlvimento e Operações
  - É o resultado de práticas técnicas, arquiteturais e culturais

O que é o DevOps
Cadeia de valor sobre a tecnologia

- Podemos associar o desenvolvimento de software à manufatura

Manufatura	**Software**
Cliente	Cliente
Matéria prima	Linhas de código
Montagem	Tempo de desenvolvimento
Entrega ao cliente	Deploy

O que é o DevOps
Cadeia de valor sobre a tecnologia

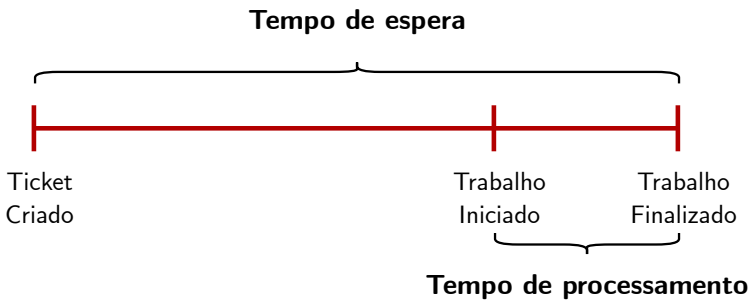
- A cadeia de valor sobre a tecnologia é **o processo** de convertermos uma hipótese de negócio em algo concreto
  1. Ideia nova
  2. Desenvolvimento
  3. Testagem
  4. Entrega
- O valor é criado **somente** quando o serviço/aplicação é entregue ao usuário final (em produção)
- Além disso, os *deployments* devem ser confiáveis e consistentes

O que é o DevOps
Lead Time

- Reduzir o tempo de espera (*lead time*) é um dos objetivos do DevOps
- Aumenta o valor final da aplicação
- Determinar o *lead time* permite mensurar a qualidade do trabalho
- O trabalho deve ser reduzido em lotes pequenos

O que é o DevOps
Processing Time

- O tempo de processamento define quanto tempo é gasto em uma tarefa
- O tempo de espera é o acúmulo da espera e o processamento
- Além disso, o **tempo de espera** é o que o usuário experiencia



Agenda

1. O que é o DevOps
2. **Princípios**
3. Atividade Prática
4. Microserviços

Princípios
Processing Time

- A **cadeia de valor** de tecnologia depende de uma movimentação "suave" de responsabilidades Ideia $\rightarrow$ Desenvolvimento $\rightarrow$ Operações $\rightarrow$ Produção
- Otimizamos o fluxo sobre esse objetivo em comum de "suavidade"
  - Taxas de conclusão
  - Taxas de problemas resolvidos/encontrados
  - Taxas de disponibilidades de operações
  - ...

Princípios
Processing Time

- Existem três conjuntos de princípios que regem o DevOps
  1. Princípios de fluxo
  2. Princípios de *feedback*
  3. Princípios de experimentação e aprendizado contínuo

Princípios
Princípios de fluxo

- Tornar visível
- *Work in Progress* (WIP)
- Reduzir tamanho dos lotes
- Reduzir transferência de informações
- Identificar e melhorar restrições

Princípios
Princípios de fluxo – Tornar visível

- Difícil visibilidade por ser um trabalho majoritariamente intelectual
- Sistematizar o fluxo de trabalho permite maior visibilidade
  - É possível identificar desperdício de tempo
  - Quais atividades estão em desenvolvimento/em *backlog*
  - Melhor priorização de atividades entre equipes

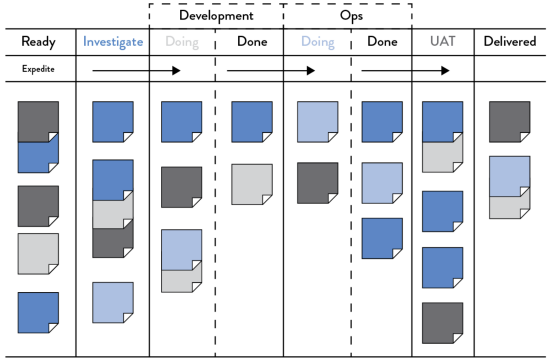


Figura: Fonte: Kim et al. (2016 apud Andersen e DeGrandis, 2012).

Princípios
Princípios de fluxo – Work in Progress

- Existe uma limitação fisiológica dos colaboradores
  - Um gerente que está associado a múltiplos processos precisa realizar uma troca de contexto entre cada uma
  - Custo de resgatar o contexto de uma determinada tarefa
- Multitarefas prejudica o tempo de término de uma tarefa, por mais simples que elas sejam!
- Utilizando um kanban podemos melhorar a gestão do tempo e limitar o trabalho em processo (WIP)
  - Exemplo: limitar para até três cartões para testes e realizar um por vez

Princípios
Princípios de fluxo – Reduzir tamanho dos lotes

- O desenvolvimento de software/manufatura trabalham com grandes lotes de produto
- Para o DevOps o objetivo é trabalhar com lotes pequenos e rápidos de serem "despachados"

- Qual o problema dos grandes lotes?
  - Alterações com efeitos "catastróficos"
  - Incorrem em mais custos
  - Aumenta WIP (piora qualidade)
  - O que fazer quando descobrimos um erro causado na primeira etapa, na última etapa?



Princípios
Princípios de fluxo – Reduzir transferência de informações

- Em muitos cenários, muito tempo é dedicado à comunicação
  - Documentações
  - Especificações
  - E-mails
  - Priorização, agendamento
  - ...
- O trabalho pode ter o contexto perdido
- Exemplo: Administrador de sistemas recebe um ticket para cadastro de usuário
  - Qual aplicação ele deve cadastrar o usuário?
  - Para qual(ais) serviço(s)?
  - Por que o acesso deve ser criado?
  - Existem dependências?

Princípios
Princípios de fluxo – Identificar e melhorar restrições

- Tempo de entrega e qualidade são fundamentais para o fluxo de valor
- Dificuldades/desperdícios que prejudicam um software:
  - Trabalho parcialmente pronto
  - Processos extras
  - Funcionalidades extras
  - Multitasking
  - Espera
  - Movimento
  - Defeitos
  - Trabalho manual ou não padronizado

Princípios
Princípios de *feedback*

- Erros em sistemas complexos são catastróficos
  - Colaborador sofrendo acidente
  - Colapso de um reator nuclear
- Nesses cenários os erros são descobertos após muito tempo ou grande acúmulo de erros
- Em software conseguimos feedback imediato
- Permite detectar problemas menores, com maior frequência

Princípios
Princípios de *feedback*

- Quanto mais rápido percebemos os problemas, mais rápido podemos resolvê-los
- Exemplo: Modelo de desenvolvimento em cascata
  - Código desenvolvido durante 1 ano
  - Sem feedback de qualidade até chegar na etapa de testes
  - Sem feedback até chegar no usuário final!
- Feedbacks demorados e esporádicos dificultam a prevenção de problemas
- Como resolver?

Princípios
Princípios de *feedback*

- Em DevOps buscamos algumas estratégias automatizadas
  - Integração
  - Build
  - Testes
- Com telemetria conseguimos observar o estado de operação dos compomentes do sistema

Princípios
Princípios de *feedback*

- O ideal é que os desenvolvedores também foquem na qualidade
- Setores de Cibersegurança e QA desenvolvem testes, porém os deixam prontos para os Devs executarem

Princípios
Princípios de experimentação e aprendizado contínuo

- Quando obtemos um feedback, é esperado que seja feita uma melhoria (se necessário)
- Como ficam as equipes que são rígidas aos próprios processos?
- Aprendizado baseado em sucessos e falhas
- Com isso aumentamos a flexibilidade e adaptabilidade dos times

Agenda

1. O que é o DevOps
2. Princípios
3. **Atividade Prática**
4. Microserviços

Atividade Prática

Objetivo

Implementar CRUD completo de tarefas com FastAPI

Endpoints

- `GET /tarefas`: Lista todas as tarefas (somente `id` e `titulo`)
- `GET /tarefas/{id}`: Visualizar tarefa específica
- `POST /tarefas`: Cria tarefa
- `PUT /tarefas{id}`: Atualiza tarefa específica
- `DELETE /tarefas/{id}`: Exclui tarefa

Estrutura da tarefa

```
{
    "id": int,
    "titulo": str,
    "descricao": str,
    "concluido": bool,
    "criado_em": datetime
}
```

Agenda

1. O que é o DevOps
2. Princípios
3. Atividade Prática
4. **Microserviços**

Microserviços

- Integração e Implantação Contínuas (CI/CD)
  - Permitem eficiência e segurnaça entre os times de desenvolvimento e operações
- Em pequenos ambientes isso é verdade, mas e quando a demanda aumenta?
  - Tráfego
  - Dados
  - Usuários
  - Leis
  - Desenvolvedores
  - Times
  - Produtos
  - Bugs
  - ...
- O principal problema é escalabilidade
- Como resolvemos?
  - Segmentando a arquitetura
  - Segmentando ambientes
  - Segmentando a base de código

Microserviços
Segmentação de arquitetura

- A arquitetura de **microsserviços** é amplamente utilizada em DevOps
  - Torna componentes do software independentes entre si
  - Facilita escalabilidade, manutenção, flexibilidade e resiliência

Microserviços
Segmentação de arquitetura

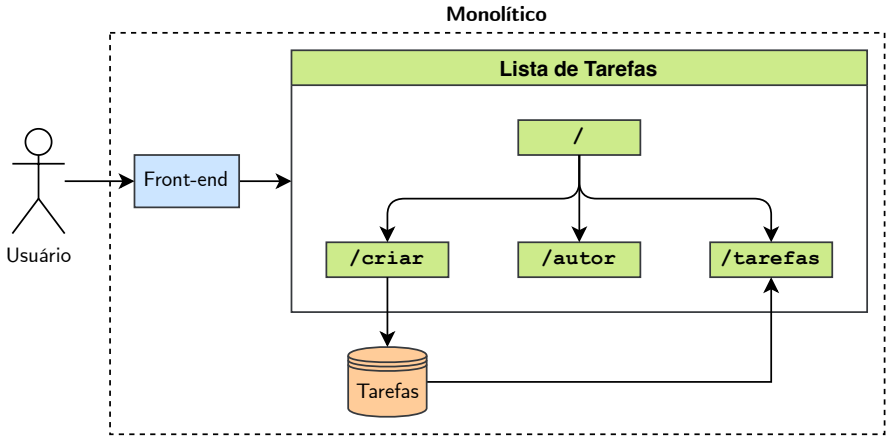


Figura: Exemplo de aplicação com arquitetura monolítica.

Microserviços
Segmentação de arquitetura

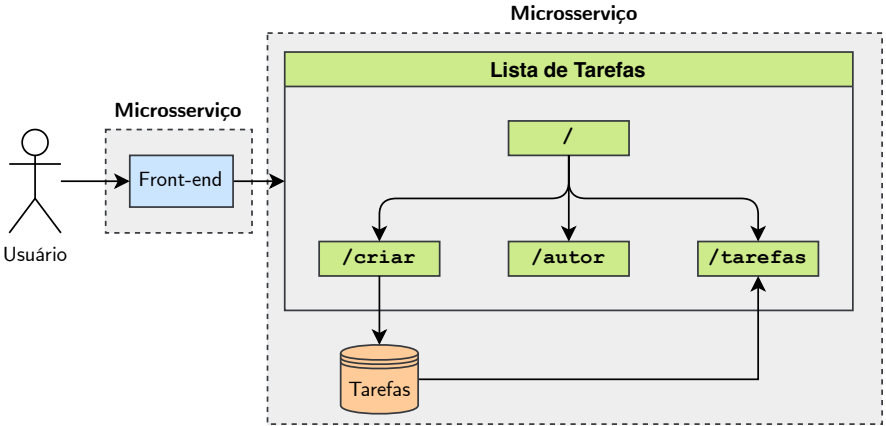


Figura: Exemplo de aplicação com arquitetura orientada a microsserviços.

Microserviços
Segmentação de arquitetura

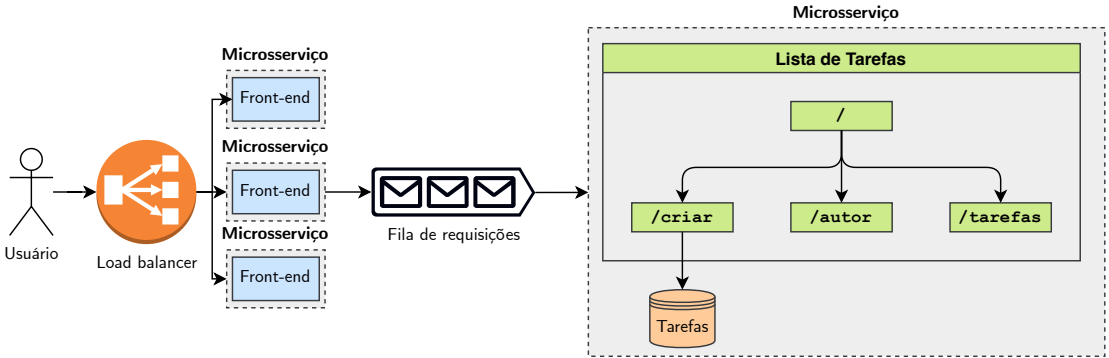


Figura: Exemplo de aplicação com arquitetura orientada a microsserviços.

Microserviços
Segmentação de arquitetura

- Existem dois tipos:
  - Nativos de nuvem
  - Reativos

Nativos de nuvem

- Os microsserviços são alocados em ambientes de nuvem (AWS, Heroku, GCP, Azure, ...)
- Cada microsserviço pode estar alocado em uma ou mais ambientes
- Todos os componentes executam em **contêineres**

Reativos

- São ativados através de mensagens
- Ficam "escutando" uma fila de mensagens até que chegue algum dado
- Focam na execução independente e assíncrona
- Representam um **processo**

Microserviços
Segmentação de arquitetura

- Para definir um microsserviço precisamos:
  1. Analisar a história do usuário
  2. Separar a história em casos de uso
- **Analisando história do usuário:** Como? O quê? Por quê?
- **Separar história em casos de uso:** Lista de funcionalidades extraídas a partir da etapa anterior

1. História do usuário

"Eu quero uma aplicação que seja capaz de gerenciar as minhas tarefas. Como se fosse um bloco de notas simples, acessível tanto do computador quanto do celular."

2. Separar história em casos de uso

Lista de funcionalidades:

- Apresentar uma página web ao usuário onde ele consegue cadastrar suas tarefas
- Armazenar todas as tarefas (pendentes e finalizadas)

Microserviços
Segmentação de arquitetura – Exemplo

Visão geral	Apresentar uma página web ao usuário onde ele consegue cadastrar suas tarefas
Ator	Usuário acessando a página web
Trigger	O usuário preenche os dados do formulário
Etapas	O usuário preenche o formulário Clica no botão "Criar" A página armazena a tarefa em um banco de dados A página web é atualizada com uma lista de todas as tarefas
Resultado	A página atualizada com todas as tarefas do usuário

Microserviços
Segmentação de arquitetura

- As aplicações baseadas em microsserviços dependem de interfaces de comunicação (APIs)
  - RESTful
- Permitem a interoperabilidade e flexibilidade entre tecnologias distintas
- Simplicidade de implementação e independente de plataforma

Microserviços
Segmentação de arquitetura

- Definimos a arquitetura RESTful com dois componentes principais:
  1. Recursos (Uma tarefa dentro do sistema)
  2. *Endpoints* (`/tarefas/tarefa-01`)
- Todas as requisições nas APIs RESTful são sobre o protocolo HTTP

Microserviços
Prática

Objetivo
Transformar a aplicação em microserviços

Arquitetura

